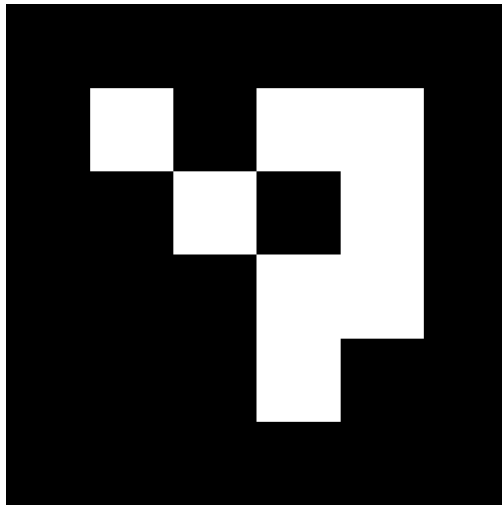


Code	HJ1
Topic	Estimation of a current drone position based on ArUco markers
Teacher	Henryk Josiński
Description	The position estimation process is based on establishing correspondences between points in the real environment (represented by ArUco markers with known 3D coordinates) and their 2D projections in the image plane. For each frame from a video sequence recorded by a drone camera: (i) detect and identify the ArUco markers, (ii) estimate the drone position (i.e., the drone-mounted camera position) using the camera matrix.
• Literature	• S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez: Automatic generation and detection of highly reliable fiducial markers under occlusion. Pattern Recognition 47, 6, pp. 2280-2292, 2014. DOI=10.1016/j.patcog.2014.01.005 • A. Marut, K. Wojtowicz, K. Falkowski: ArUco markers pose estimation in UAV landing aid system. IEEE, 2019. • M. F. Sani: Automatic navigation and landing of an indoor AR drone quadrotor using ArUco marker and inertial sensors (conference paper), 2017. • Documentation of the ArUco module from the OpenCV library.
Group	Krzysztof Żywko Jakub Biernat Kamil Mutwin Łukasz Idziak

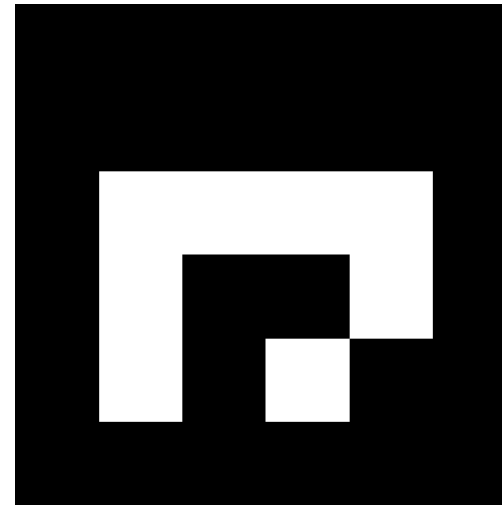
ArUco markers

- An ArUco marker is a synthetic square pattern consisting of a wide black border and an inner binary matrix that encodes its identifier (ID). The structure of the binary matrix is defined by the chosen marker dictionary (e.g., 4×4, 5×5), whereas the physical marker size determines the scale of the pattern in the image.
- Examples of 4-by-4 markers:

id = 0



id = 1

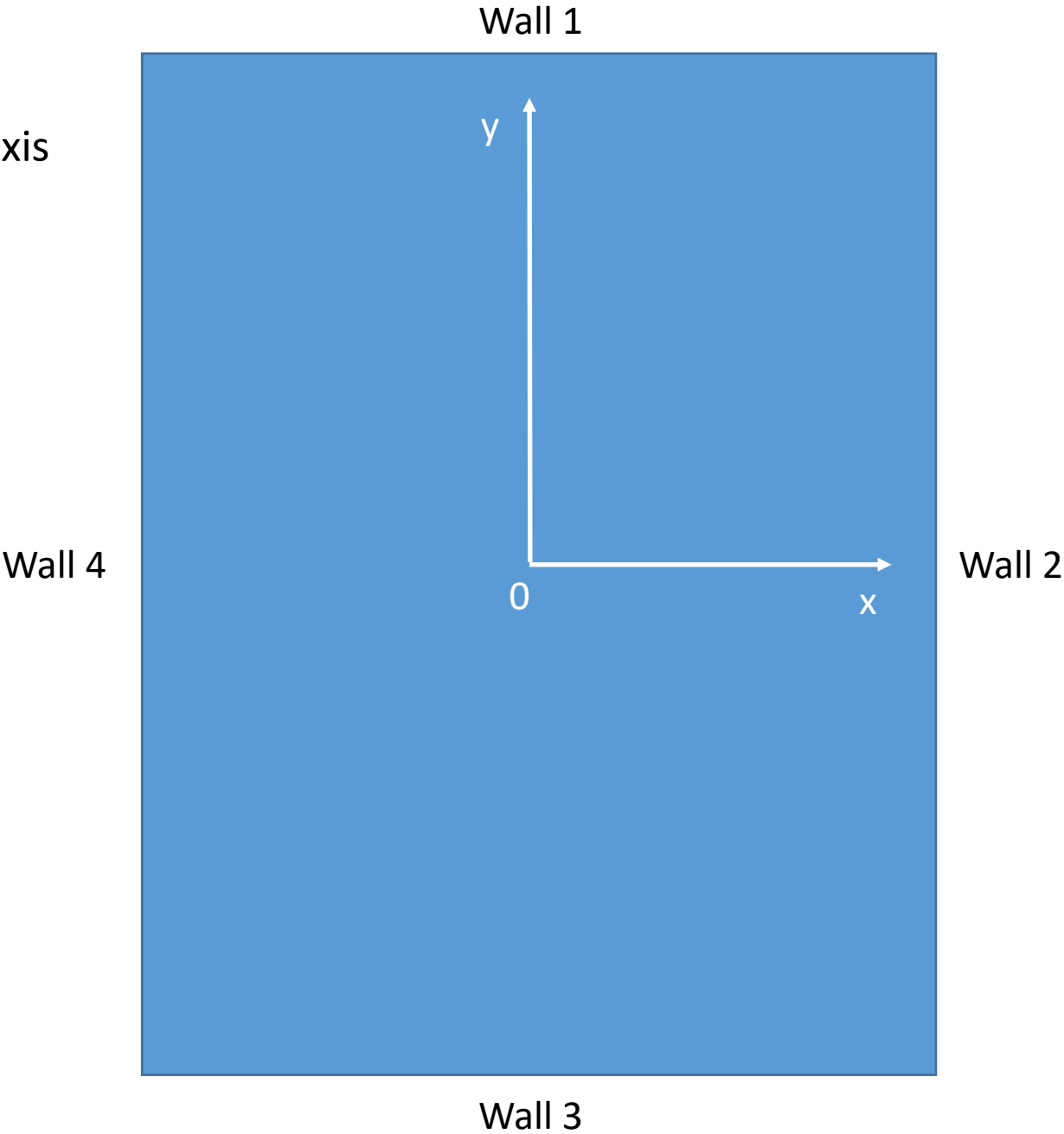


- The black border facilitates its fast detection in the image, while the binary encoding enables identification and the application of error detection and correction techniques.
- A marker dictionary (e.g., DICT_4X4_1000) defines a predefined set of markers characterized by a fixed matrix size and a specified number of unique identifiers, designed for use in a given application.

Goal of the project

- Assumption: while a drone is flying in a room with ArUco markers on the walls, the drone camera is recording the surroundings.
- How to estimate the current drone position?
- The position estimation process is based on establishing correspondences between points in the real environment (represented by ArUco markers with known 3D coordinates of their corners – here: the upper-right corner) and their 2D projections in the image plane.
- For each frame from a video sequence recorded by the drone camera with a frequency of 25 Hz:
 - 1) Detect and identify the ArUco markers,
 - 2) Estimate the drone position (i.e., the 3D coordinates of the camera position) using the camera intrinsic parameters (the intrinsic matrix), and both 2D and 3D coordinates of the correctly identified ArUco markers,
 - 3) Estimate the error based on the distance between the calculated drone position and the reference position read from the csv file (the csv file consists of the 3D coordinates of the special markers mounted on the drone, the trajectories of which were recorded using the motion capture (mocap) technique with a frequency of 100 Hz).

Vertical view of the
laboratory room (the z -axis
points vertically up)



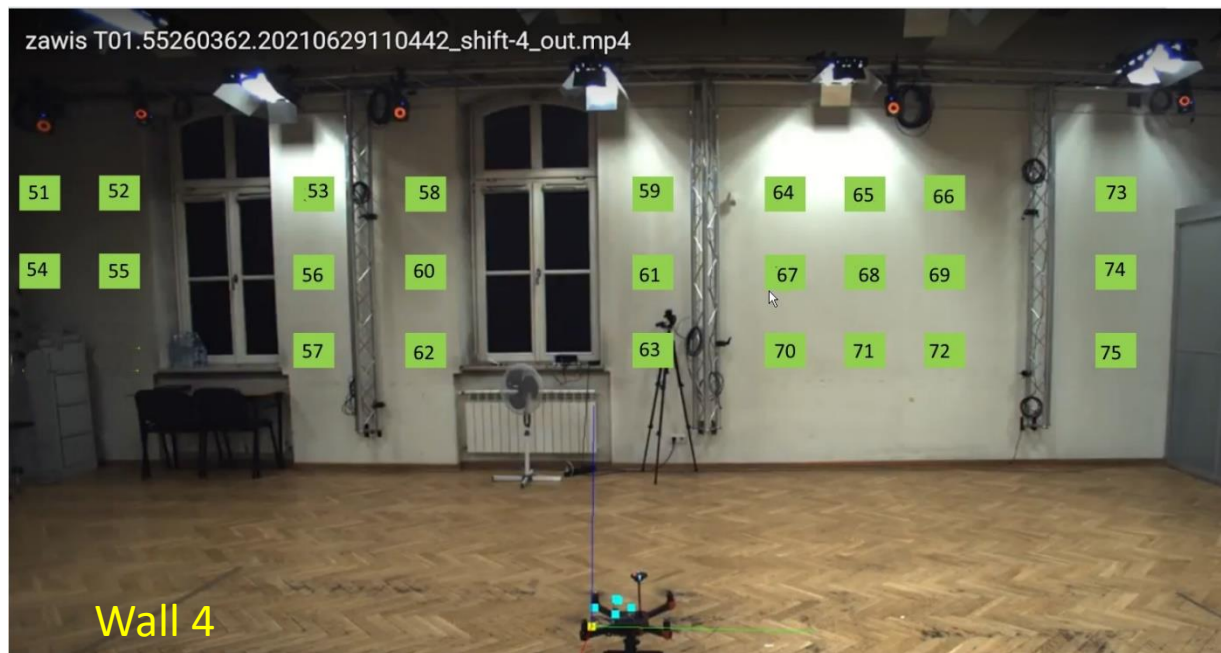
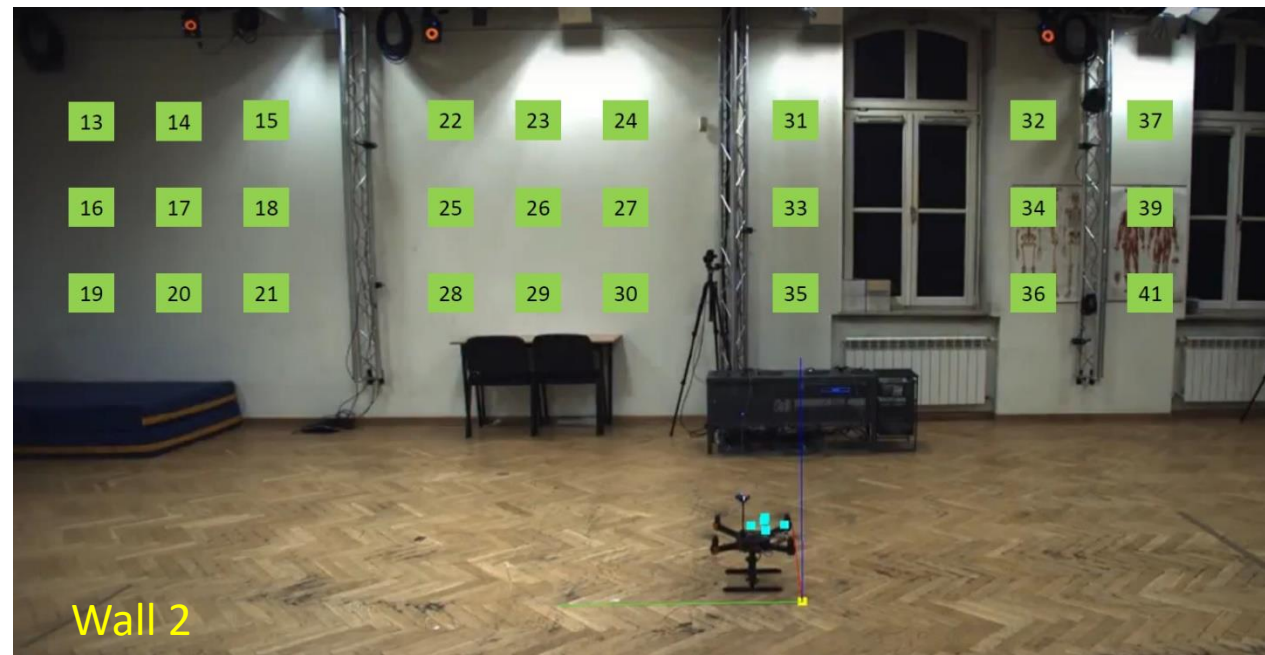
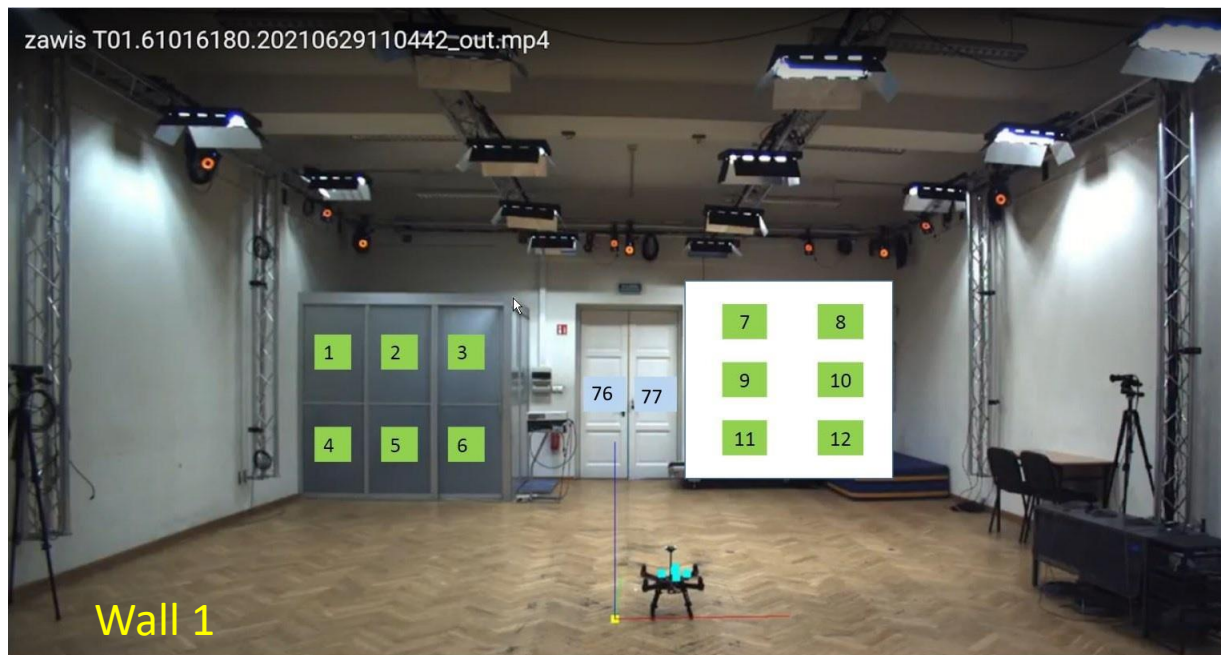


Photo Wall 2 does not show markers 38, 40, 42, which are further to the right!

Algorithm

- Create the matrix of camera intrinsic parameters (the intrinsic matrix), which consists of the following parameters (based on two calibration variants):
 - focal length
 - principal point
 - image size
- For each frame of the video sequence:
 - read the frame
 - detect and identify the ArUco markers $\Rightarrow$ obtain marker IDs and 2D image coordinates
 - read the 3D coordinates of the upper-right corners from the Excel spreadsheet
 - estimate the current camera position based on the intrinsic matrix and the corresponding 2D and 3D coordinates of the identified ArUco markers
 - estimate the error as the distance between the calculated drone position and the reference position read from the csv file

Input data

- Video sequences from a drone camera: GX010280.mp4, GX010282.mp4, GX010290.mp4

- Image size: 3840×2160

- Camera intrinsic parameters:

Variant 1

- focal length: [1840, 1840] [pixel]
- principal point: [1920, 1080] [pixel]
- field of view: horizontal 92.45 [°], vertical 60.83 [°]
- radial distortion: [0, 0, 0]
- tangential distortion: [0, 0]



Input data

Variant 2 (based on another calibration method)

Camera Matrix:

```
[[1.83319394e+03  0.00000000e+00  1.93840794e+03]
 [0.00000000e+00  1.83174273e+03  1.07227436e+03]
 [0.00000000e+00  0.00000000e+00  1.00000000e+00]]
```

Distortion Coefficients: [0.00156918 -0.00406143 -0.00167778
0.00183561 0.00387331]

HFOV: 92.65 **VFOV:** 61.01

The order of distortion coefficients as in OpenCV: k_1, k_2, p_1, p_2, k_3

- ArUco markers:
 - identifiers and 3D coordinates: [ArUco_markers_3D.xlsx](#) [Marker_ID, X, Y, Z, Wall]
 - dictionary: ['4x4_1000'](#)
 - range of identifiers: [1:83](#)
- Mocap reference data: [GX010280_U.csv](#), [GX010282_obroty.csv](#), [GX010290_z_reki.csv](#)

Reference data

Four mocap markers (the x, y, z coordinates of the mocap markers in a single row of the csv file are

- for marker 1: elements 3,4,5;
- for marker 2: elements 6,7,8;
- for marker 3: elements 9,10,11;
- for marker 4: elements 12,13,14)

- Make the initial correction using marker 3 [mm]:

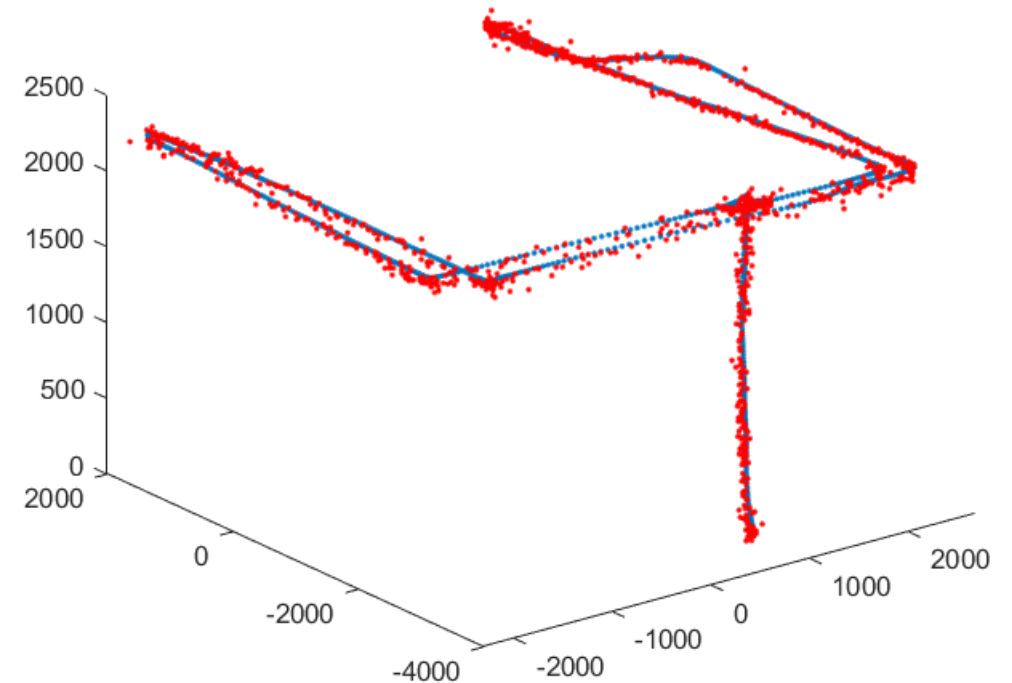
$$\begin{aligned}x_{cam} &= x_{m3} + 2 \\y_{cam} &= y_{m3} + 5 \\z_{cam} &= z_{m3} - 25\end{aligned}$$

- The distance between the marker and the camera remains constant, while the distances in individual axes ($\Delta x, \Delta y, \Delta z$) change
- Take into account the difference in frequencies (100 Hz vs. 25 Hz)!
- Start from row: 455 (GX010280), 82 (**GX010282**), 75(**GX010290**)!



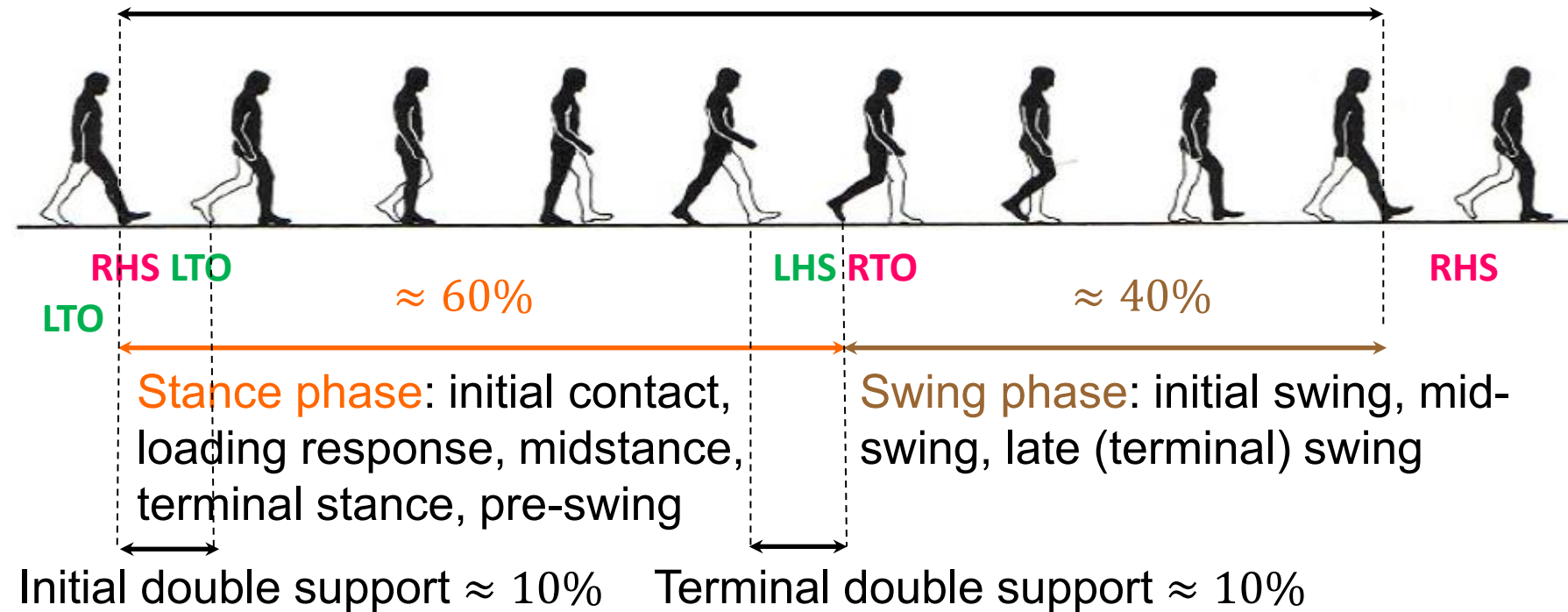
Expected results

- A matrix of intrinsic parameters of the camera
 - A drone position (equivalent to the camera position) for each frame of a video sequence $\Rightarrow$ a drone trajectory
 - A total error estimation based on the distance between the corresponding points on the calculated trajectory and on the trajectory of drone's marker 3 for consecutive frames
 - Immediate rejection of an incorrectly determined position(i.e., without using reference data)
- Hint: the ArUco module from the OpenCV library



Code	HJ2
Topic	Video-Based Estimation of Human Gait Cycle Parameters
Teacher	Henryk Josiński
Description	Assumptions: • input data: three side-view gait sequences (avi files): T01, T02, T03 • static camera ($f = 50$ Hz) • single walking subject • the subject's height is 181 cm (anthropometric calibration) • reference data: c3d files with heel-strike (HS) and toe-off (TO) events • one gait cycle defined as HS-to-HS of the same foot Key information: • HS and TO events Approaches: • silhouette-based • pose estimation (OpenPose (??)) • deep learning (?) ← insufficient amount of data Potential problems: • occlusion – some markers become partially or completely hidden from the camera Expected results: • gait cycle duration • step length and stride length • cadence (steps per minute) • walking speed • stance and swing phase durations • joint trajectories over time (optional)
Literature	Stenum J, Rossi C, Roemmich RT (2021) Two-dimensional video-based analysis of human gait using pose estimation. PLoS Comput Biol 17(4): e1008935. https://doi.org/10.1371/journal.pcbi.1008935
Group	Ahmad Gamidi Uladzislau Rakhuba Henok Urufa

Gait cycle



CAREN Extended System (Computer Assisted Rehabilitation Environment)



- a dome with an embedded dual belt treadmill mounted onto a platform with six degrees of freedom
- a virtual reality environment
- an integrated motion capture system
- visual and audio stimuli
- a safety harness
- GRF and EMG measurement